

KAUNO TECHNOLOGIJOS UNIVERSITETAS
INFORMATIKOS FAKULTETAS

Modulio P170B328 Lygiagretusis programavimas
Inžinerinis projektas

Dėstytojai: Kiudys Eligijus
asist. JANČIUKAS Mindaugas

Studentas: Karolis Žilinskas IFF-3/10

Contents

Užduotis:	3
Analizė ir sprendimo metodas:	3
Testavimas ir programos vykdymo instrukcija:	3
Vykdymo laiko kitimo tyrimas:	4
a.	4
b.	4
Grafikai:	7
c. aprašymas:	12
Išvados:	14

Užduotis:

Miestas išsidėstęs kvadrato, kurio koordinatės $(-10 \leq x \leq 10, -10 \leq y \leq 10)$. Mieste yra n ($n \geq 3$) vieno tinklo parduotuvių, kurių koordinatės yra žinomos (*Koordinatės gali būti generuojamos atsitiktinai, negali būti kelios parduotuvės toje pačioje vietoje*). Planuojama pastatyti dar m ($m \geq 3$) šio tinklo parduotuvių. Parduotuvės pastatymo kaina (vietos netinkamumas) vertinama pagal atstumus iki kitų parduotuvių ir miesto ribos. Reikia parinkti naujų parduotuvių vietas (koordinates) taip, kad parduotuvių pastatymo kainų suma būtų kuo mažesnė.

Atstumo tarp dviejų parduotuvių, kurių koordinatės (x_1, y_1) ir (x_2, y_2) , kaina apskaičiuojama pagal formulę:

$$C(x_1, y_1, x_2, y_2) = \exp(-0.1 \cdot ((x_1 - x_2)^2 + (y_1 - y_2)^2))$$

Atstumo tarp parduotuvės, kurios koordinatės (x_1, y_1) , ir artimiausio miesto ribos taško, kurio koordinatės (x_r, y_r) , kaina apskaičiuojama pagal formulę:

$$C^R(x_1, y_1, x_r, y_r) = \begin{cases} 0, & \text{jeigu parduotuvę planuojama statyti miesto ribose} \\ 0.5 \cdot ((x_1 - x_r)^2 + (y_1 - y_r)^2), & \text{kitais atvejais} \end{cases}$$

Analizė ir sprendimo metodas:

Problemai spręsti reikia naudoti kokį nors gradientinį metodą (naudoas gradientienis nusileidimas). Išlygegretinta pagal pasirinktą didžiausią gijų kiekį, jis bus neviršijamas, bet yra 3 metodai kurie gali dirbti vienu metu. Gradient(), CalcTotalCost() ir Norm(). Pagrindinis ciklas iškviės pasirinktą Gradient() kiekį ir kai jis uzsibaigs bus tiek pat lygegrečių Norm() metodų. CalcTotalCost() taip pat kartais dirbs vienu metu, bet kai yra kviečiamas iš Gradient() nesukurs papildomų gijų.

Testavimas ir programos vykdymo instrukcija:

Programa veikia teisingai, randa optimalesnį nei pradinį parduotuvių išsidėstymą. Programa prasideda prašant pasirinkti gijų kiekį. Kiekį pasirinkti, paspausti enter ir programa bus įvykdyta.

Vykdomo laiko kitimo tyrimas:

a.

Paruošti bent 8 skirtingos apimties duomenų rinkinius.

Duomenų rinkinys bus kuriamų naujų parduotuvių skaičius.

Visi rinkiniai:

1	2	4	8	16	32	64	128
---	---	---	---	----	----	----	-----

b.

Procesų laikai priklausant nuo gijų skaičiaus (ms):

1 gija:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
92	125	204	401	1047	3414	11974	42795	157036

2 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
104	151	232	409	905	2554	7932	2651	94756

3 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
109	159	229	429	834	2143	6431	21018	74955

4 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
114	171	219	354	827	2116	6134	20347	73256

5 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
119	169	233	358	735	1848	5530	19921	74334

6 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
120	176	240	356	735	1650	5101	17544	7319

7 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
121	180	239	349	751	1794	4897	17998	69349

8 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
129	187	231	348	689	1492	4458	15987	63511

9 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės

135	184	249	375	705	1601	4896	16112	61378
-----	-----	-----	-----	-----	------	------	-------	-------

10 gijų:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
135	189	245	383	701	1550	4693	14520	58974

16 gijų:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1487	4075	14442	51011

24 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1537	4102	12445	45315

32 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1543	4220	13254	45077

40 gijų:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1550	4551	12349	43626

48 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1564	4545	12987	44832

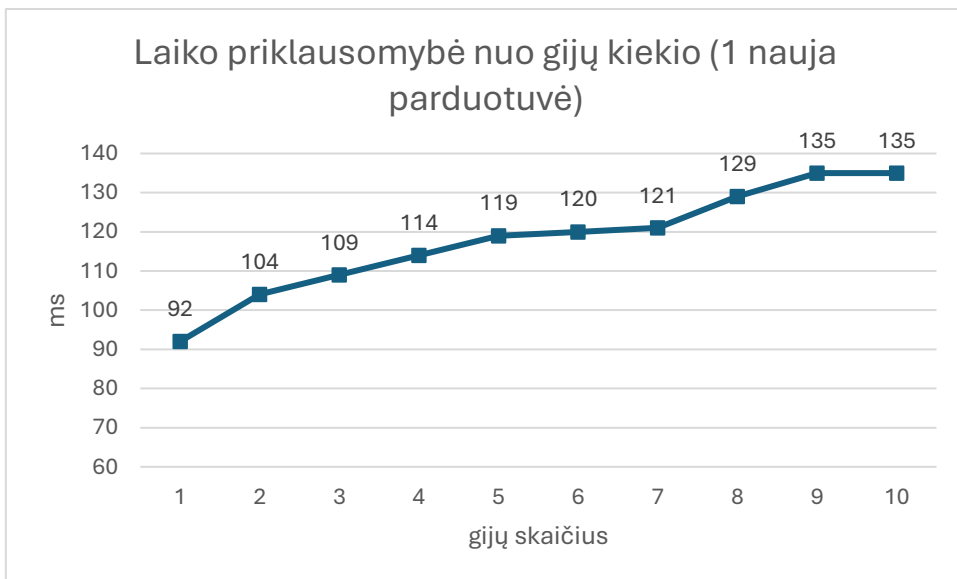
56 gijos:

1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1542	4612	12748	44457

64 gijos:

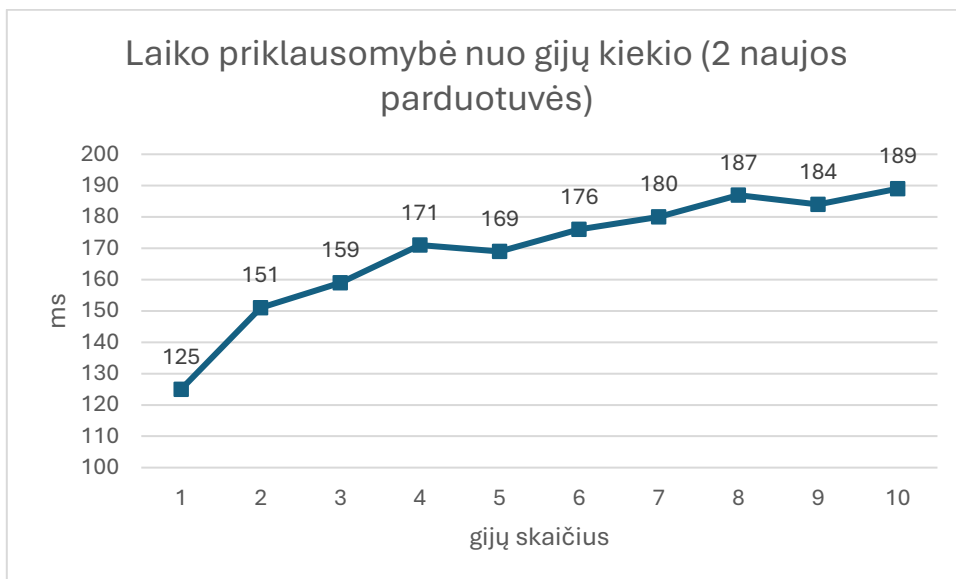
1 nauja parduotuvė	2 naujos parduotuvės	4 naujos parduotuvės	8 naujos parduotuvės	16 naujų parduotuvių	32 naujos parduotuvės	64 naujos parduotuvės	128 naujos parduotuvės	256 naujos parduotuvės
-	-	-	-	-	1699	4697	12947	45965

Grafikai:



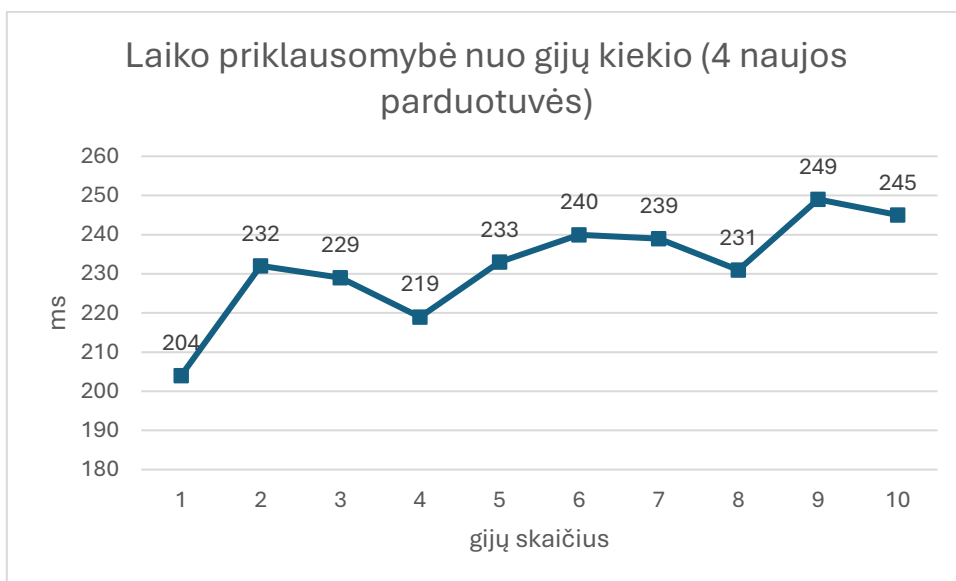
Optimalus gijų kiekis: 1.

Vykdomo laikas kilo didinant gijų kiekį



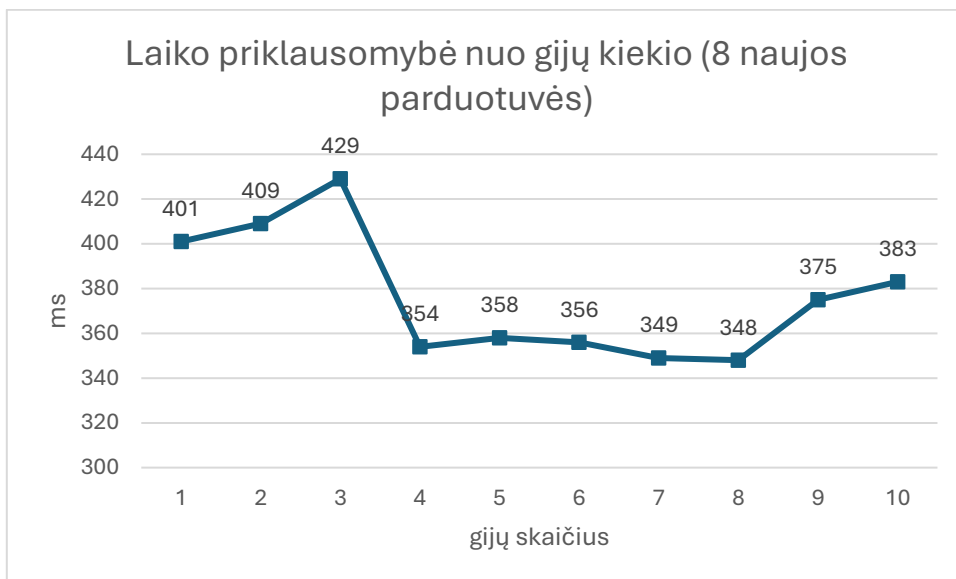
Optimalus gijų kiekis: 1.

Vykdymo laikas kilo didinant gijų kiekį



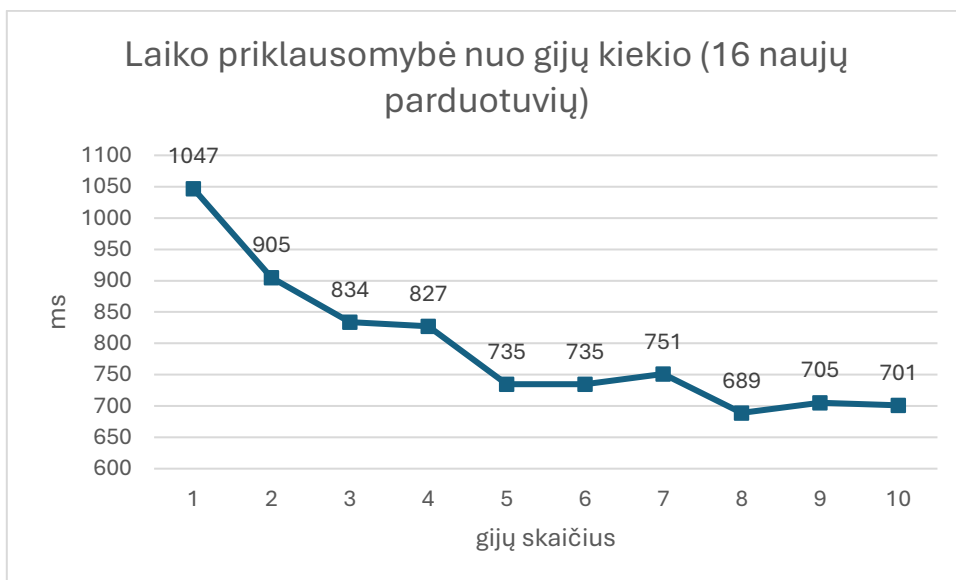
Optimalus gijų kiekis: 1.

Vykdymo laikas kilo didinant gijų kiekį, bet nežymiai



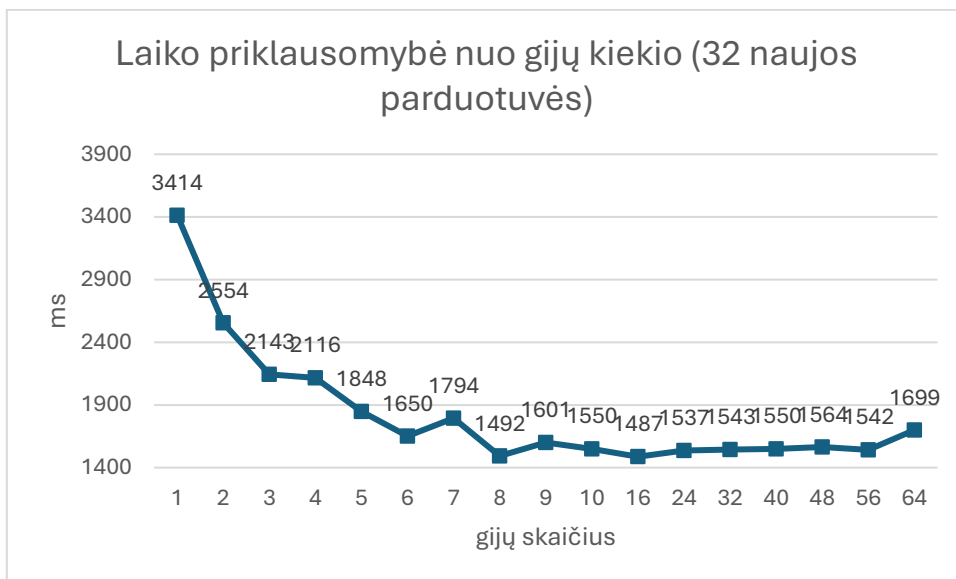
Optimalus gijų kiekis: 4-8.

Vykdymo laikas iš pradžių krito didinant gijų kiekį, bet vėliau pradėjo augti



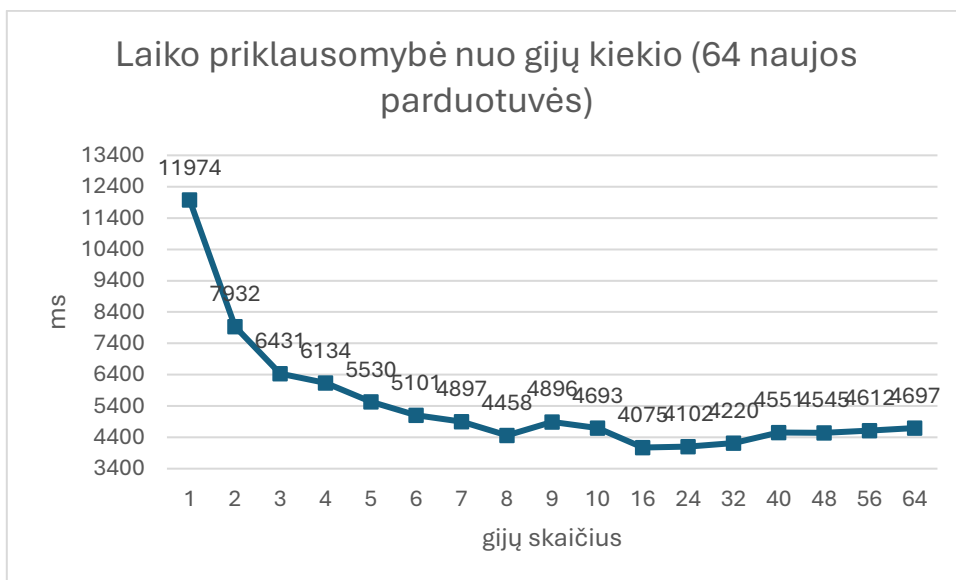
Optimalus gijų kiekis: 8.

Vykdymo laikas krito didinant gijų kiekį, bet nežymiai, ypač mažas vykdymo laiko pokytis prasidėjo nuo penktos gijos



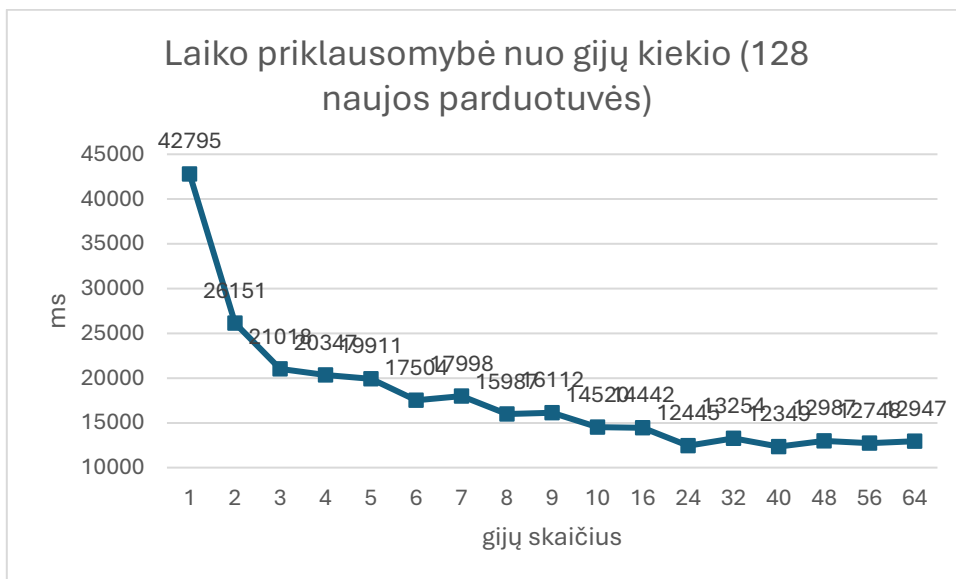
Optimalus gijų kiekis: 8.

Vykdymo laikas krito didinant gijų kiekį, ir kito mažai nuo aštuntos gijos



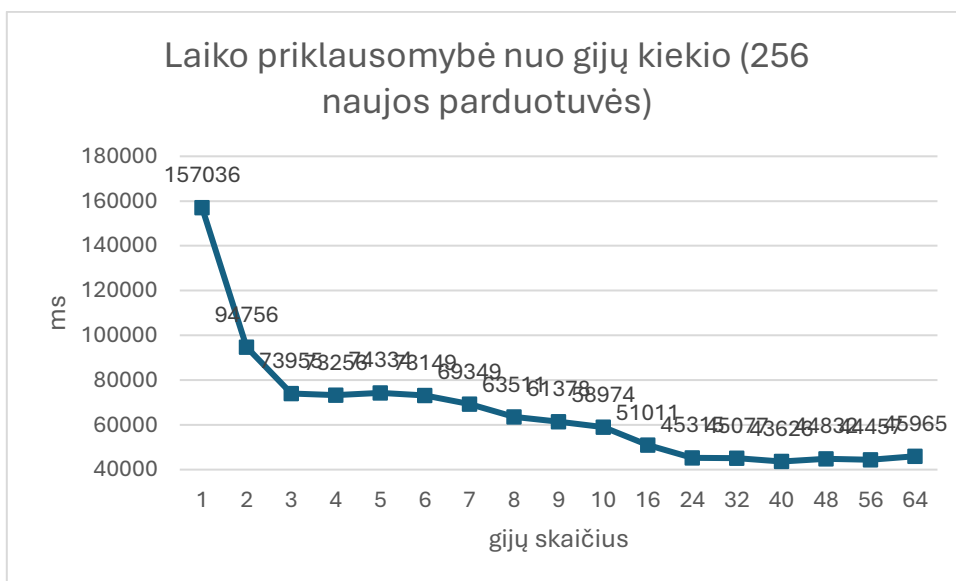
Optimalus gijų kiekis: 16.

Vykdymo laikas krito didinant gijų kiekį, pamažu pradėjo kilt po 16tos gijos



Optimalus gijų kiekis: 24.

Vykdymo laikas krito didinant gijų kiekį ir išsilygino po 24-tos gijos



Optimalus gijų kiekis: 24.

Vykdymo laikas krito didinant gijų kiekį ir išsilygino po 24-tos gijos.

c. aprašymas:

Manau mažesni duomenų kiekiai dirbo greičiau su 1 gija nei su keliomis, nes atlikti nedaug darbo trunka trumpiau nei kurti ir valdyti papildomas gijas, o didieji duomenų kiekiai galėjo optimaliau išnaudoti teikiamą gijų kiekį.

Pagreitėjimas kiekvienam duomenų kiekiui lyginant su savo optimaliu gijų skaičiumi:

1 parduotuvė	2 parduotuvės	4 parduotuvės	8 parduotuvės	16 parduotuvių	32 parduotuvės	64 parduotuvės	128 parduotuvės	256 parduotuvės
1	1	1	1.152299	1.519594	2.295898	2.938405	3.465463	3.599597

Jei pagreitėjimas lygus vienam, tai reiškia, kad programos greitis naudojant gijų kiekį, didesnis nei 1, lėtėjo.

Sakykim apytiksliai paralelizavotos programos dalys, palei naujų parduotuvių kiekį, kur P – procentali paralelizavota dalis, S – neparalelizavota yra:

	1 parduotuvė	2 parduotuvės	4 parduotuvės	8 parduotuvės	16 parduotuvių	32 parduotuvės	64 parduotuvės	128 parduotuvės	256 parduotuvės
P	55	65	75	82	86	89	91	92	94
S	45	35	25	18	14	11	9	8	6

Tai pagal Amdalo dėsnį pagreitėjimai turėtų būti:

Gijų skaičius:	1 parduotuvė	2 parduotuvės	4 parduotuvės	8 parduotuvės	16 parduotuvių	32 parduotuvės	64 parduotuvės	128 parduotuvės	256 parduotuvės
1	1	1	1	1	1	1	1	1	1
2	1.38	1.48	1.60	1.69	1.75	1.80	1.83	1.85	1.89
3	1.58	1.76	2.00	2.21	2.34	2.46	2.54	2.59	2.68
4	1.70	1.95	2.29	2.60	2.82	3.01	3.15	3.23	3.39
5	1.79	2.08	2.50	2.91	3.21	3.47	3.68	3.79	4.03
6	1.85	2.18	2.67	3.16	3.53	3.87	4.14	4.29	4.62
7	1.89	2.26	2.80	3.37	3.80	4.22	4.55	4.73	5.15
8	1.93	2.32	2.91	3.54	4.04	4.52	4.91	5.13	5.63
9	1.96	2.37	3.00	3.69	4.25	4.79	5.23	5.49	6.08
10	1.98	2.41	3.08	3.82	4.42	5.03	5.52	5.81	6.49

16						6.04	6.81	7.27	8.42
24						6.80	7.82	8.45	10.08
32						7.26	8.44	9.20	11.19
40						7.56	8.87	9.71	11.98
48						7.78	9.18	10.08	12.57
54						7.94	9.41	10.37	13.02
64						8.07	9.60	10.60	13.39

Taigi arba blogai įvertinau savo programos paralelizuojamą dalį, arba blogai apskaičiavau teorišką programos greitėjimo priklausomybę nuo gijų, arba realus atvejis neatitinka šių skaičiavimų.

Pavyzdžiui, numatoma, kad ir kai turime vieną parduotuvę, programos greitis augs didinant gijų kiekį, bet realybėje net nėra esmės kurti papildomų gijų, nes nėra su kuom jas įdarbinti, tai jų kūrimas tik apkrauna CPU.

Išvados:

Tikėjausi, kad su aukštais skaičiais duomenų, bus drastiškesnis laiko trumpėjimas keliant gijų kiekį ir kad vienintelis atvejis kai gijų kūrimas lėtins programą, bus tada, kai gijų kuriamą daugiau nei yra tikrinamą naujų parduotuvių.

Manau spręsti šiai paralelinimo problemai C# PLINQ tinka.

Programos kodas:

```
using System;

using System.Linq;

using System.Threading.Tasks;

using System.Diagnostics;

class Program
{
    static Random rand = new Random();

    static void Main()
    {
        Console.Write("kiek giju naudoti?");

        int threadCount = int.Parse(Console.ReadLine());

        Stopwatch stopwatch = new Stopwatch();
        stopwatch.Start();

        //ribos
        double x_min = -10, x_max = 10, y_min = -10, y_max = 10;

        //parduotuves
        int n_existing = 3;
        int m_new = 32;

        double[,] existingStores = RandomPoints(n_existing, x_min, x_max, y_min,
y_max);

        double[,] newStores = RandomPoints(m_new, x_min, x_max, y_min, y_max);
        double[,] initialNewStores = (double[,])newStores.Clone();

        //pradinis vertinimas
```

```

        var (initialCost, initialCosts) = CalcTotalCost(newStores, existingStores,
threadCount);

        Console.WriteLine("pradine bendra kaina: " + initialCost);

        Console.WriteLine("pradines nauju parduotuviu kainos: " + string.Join("\n",
initialCosts));

        double stepSize = 0.1;
        int iterations = 4000;
        double tolerance = 1e-6;

        double[] allCosts = new double[iterations];

        //main
        for (int iter = 0; iter < iterations; iter++)
        {
            double[,] grad = Gradient(newStores, existingStores, threadCount);

            //gradient
            for (int i = 0; i < m_new; i++)
                for (int j = 0; j < 2; j++)
                    newStores[i, j] -= stepSize * grad[i, j];

            var (cost, _) = CalcTotalCost(newStores, existingStores, threadCount);
            allCosts[iter] = cost;

            if (Norm(grad, threadCount) < tolerance)
                break;
        }

        Console.WriteLine("\nsenu parduotuviu koordinates:");

```



```

PrintMatrix(existingStores);

Console.WriteLine("\npradines nauju parduotuviu koordinates:");
PrintMatrix(initialNewStores);

Console.WriteLine("\ngalutines optimizuotu parduotuviu koordinates:");
PrintMatrix(newStores);

var (finalTotal, finalCosts) = CalcTotalCost(newStores, existingStores,
threadCount);

Console.WriteLine("\nbendra kaina po optimizacijos: " + finalTotal);
Console.WriteLine("kiekvienos parduotuves kaina po optimizacijos: " +
    string.Join("\n", finalCosts));

stopwatch.Stop();
Console.WriteLine($"laiko trukme: {stopwatch.ElapsedMilliseconds} ms");
}

static double[,] RandomPoints(int n, double xMin, double xMax, double yMin, double
yMax)
{
    double[,] arr = new double[n, 2];
    for (int i = 0; i < n; i++)
    {
        arr[i, 0] = rand.NextDouble() * (xMax - xMin) + xMin;
        arr[i, 1] = rand.NextDouble() * (yMax - yMin) + yMin;
    }
    return arr;
}

```

```

static double StoreDistanceCost(double x1, double y1, double x2, double y2)
{
    double dx = x1 - x2;
    double dy = y1 - y2;
    return Math.Exp(-0.1 * (dx * dx + dy * dy));
}

static double BoundaryDistanceCost(double x, double y,
                                   double xMin = -10, double xMax = 10,
                                   double yMin = -10, double yMax = 10)
{
    if (x >= xMin && x <= xMax && y >= yMin && y <= yMax)
        return 0;

    double xDist = Math.Min(Math.Abs(x - xMin), Math.Abs(x - xMax));
    double yDist = Math.Min(Math.Abs(y - yMin), Math.Abs(y - yMax));

    return 0.5 * (xDist * xDist + yDist * yDist);
}

/// <summary>
/// total cost
/// </summary>
/// <param name="newStores"></param>
/// <param name="existingStores"></param>
/// <param name="threads"></param>
/// <returns></returns>

static (double totalCost, double[] storeCosts) CalcTotalCost(double[,] newStores,
double[,] existingStores, int threads)
{

```

```

int m = newStores.GetLength(0);
int n = existingStores.GetLength(0);

double totalCost = 0;
double[] storeCosts = new double[m];
object lockObj = new object();

=> Parallel.For(0, m, new ParallelOptions { MaxDegreeOfParallelism = threads }, i
{
    double nx = newStores[i, 0];
    double ny = newStores[i, 1];
    double cost = 0;

    for (int j = 0; j < n; j++)
    {
        double ex = existingStores[j, 0];
        double ey = existingStores[j, 1];
        cost += StoreDistanceCost(nx, ny, ex, ey);
    }

    cost += BoundaryDistanceCost(nx, ny);
    storeCosts[i] = cost;

    lock (lockObj)
    {
        totalCost += cost;
    }
});

return (totalCost, storeCosts);
}

```

```

    static double[,] Gradient(double[,] newStores, double[,] existingStores, int
threads)
    {
        double h = 1e-5;
        int m = newStores.GetLength(0);

        double[,] grad = new double[m, 2];

        Parallel.For(0, m, new ParallelOptions { MaxDegreeOfParallelism = threads }, i
=>
        {
            for (int j = 0; j < 2; j++)
            {
                double orig = newStores[i, j];

                newStores[i, j] = orig + h;
                double fPlus = CalcTotalCost(newStores, existingStores, threads).Item1;

                newStores[i, j] = orig - h;
                double fMinus = CalcTotalCost(newStores, existingStores,
threads).Item1;

                grad[i, j] = (fPlus - fMinus) / (2 * h);

                newStores[i, j] = orig;
            }
        });

        return grad;
    }

```

```

static double Norm(double[,] matrix, int threads)
{
    return Math.Sqrt(
        Enumerable.Range(0, matrix.GetLength(0))
            .SelectMany(i => Enumerable.Range(0, matrix.GetLength(1))
                .Select(j => matrix[i, j] * matrix[i, j]))
            .AsParallel()
            .WithDegreeOfParallelism(threads)
            .Sum()
    );
}

static void PrintMatrix(double[,] arr)
{
    int m = arr.GetLength(0);
    for (int i = 0; i < m; i++)
    {
        Console.WriteLine($"[{arr[i, 0]:F4}, {arr[i, 1]:F4}]");
    }
}
}

```